Drone Simulation

User Manual

Autonomous Navigation and Obstacle Avoidance Simulation

Table of Contents

1. System Requirements & Installation
2. Project File Structure
3. Launching the Simulation
4. Environment & Coordinate System
5. Keyboard Controls Reference (Complete)
6. Standard Flight Operations (Step-by-Step)
7. Terrain Scanning System
8. Obstacle Avoidance
9. Behavior Priorities
10. Path Selection (A* / Dijkstra)
11. Sensor Filtering
12. Output Files
13. Error Handling
14. Troubleshooting Guide
15. Appendix

1. System Requirements & Installation

The simulation requires a Windows system with Python 3.10 or above. The following packages are necessary:

- **mujoco** (≥ 3.0): Physics engine and viewer.
- **numpy**: Numerical array processing.
- **Pillow (PIL)**: Image generation for terrain maps.
- **matplotlib**: Sensor filtering comparison graphs.
- **reportlab**: PDF generation for reports and this manual.

Install all dependencies via: `pip install mujoco numpy pillow matplotlib reportlab`

> ***Note:*** *The* `run.bat` *launcher script automatically checks for and installs missing* `mujoco` *and* `pillow` *packages. Other packages must be installed manually.*

---

2. Project File Structure

File	Type	Purpose
scene.xml	MuJoCo Model	Complete world definition: terrain, pads, rocks, drone, dynamic obstacles.
simulate.py	Main Entry	Simulation loop, autopilot, key handling, viewer rendering, all integrations.
sensor_simulation.py	Sensor Suite	Virtual sensors (GPS, IMU, Baro, Lidar, Front), Kalman/EMA/Comp filters.
terrain_scanner.py	Feature 1	Onboard terrain scanning with directional/omni sensor and Fog-of-War grid.
dynamic_obstacles.py	Feature 2	Runtime obstacle spawning, path blocking, and occupancy grid updates.
subsumption.py	Feature 3	Behavior priority manager: Kill Switch, Avoidance, Navigation, Scanning.
generate_assets.py	Asset Gen	Generates neon grid texture and rough terrain heightmap PNG.
add_rocks.py	Asset Gen	Procedurally generates 50 stone obstacles into scene.xml.
generate_pdf.py	Report Gen	Generates the detailed project report PDF with benchmarks.
generate_manual_pdf.py	Manual Gen	Generates this user manual PDF.
run.bat	Launcher	One-click launcher: checks deps, generates assets, runs simulation.

Generated output files:

- **ground_map.png**: 2D occupancy map with Fog-of-War overlay showing explored / unexplored regions.
- **shortest_path_map.png**: Same as above but with the planned path drawn on top.
- **map_data.json**: Full serialised occupancy grid, landing spots, path waypoints, and metadata.
- **occupancy_grid.csv**: Raw 160×160 grid (0=free, 1=obstacle, -1=unknown).
- **flight_trajectory.csv**: Timestamped position, quaternion, velocity, and forces log.
- **detection_log.csv**: Flight events (takeoff, scan, pad arrival, landing, kill switch).
- **sensor_filtering_analysis.png**: 6-panel comparative analysis graph saved on simulation exit.

3. Launching the Simulation

Method 1 — run.bat (Recommended): Double-click `run.bat`. It runs `generate_assets.py` (grid texture + heightmap), then `add_rocks.py` (50 stone obstacles into scene.xml), then launches `simulate.py`.

Method 2 — Manual: Open a terminal in the project directory and run:

```
python generate_assets.py
python add_rocks.py
python simulate.py
```

> ***Important:*** *You must click inside the MuJoCo viewer window after it opens to capture keyboard input. Without clicking, keypresses are not forwarded.*

---

4. Environment & Coordinate System

The arena spans **16 m × 16 m** (X from –8.0 to +8.0, Y from –8.0 to +8.0). The terrain is a rough Martian/Lunar heightfield with flattened circles at each landing pad. Six landing pads are pre-configured:

Pad Name	World X	World Y	Radius (m)	Key Binding
Home	–4.0	–4.0	0.6	H
Pad 1	–4.0	4.0	0.4	1
Pad 2	4.0	4.0	0.4	2
Pad 3	4.0	–4.0	0.4	3
Pad 4	0.0	2.0	0.4	4
Pad 5	0.0	–2.0	0.4	5

The occupancy grid maps this 16 m² world into a **160 × 160** integer grid. Each cell is approximately **0.1 m × 0.1 m**. Cells with value 1 are obstacles; value 0 are free; value –1 are unexplored (Fog of War).

5. Keyboard Controls Reference (Complete)

All keys are case-insensitive. You must click inside the viewer window first.

Key	Function	Required State	Detailed Behaviour
L	Takeoff / Land	Any	If LANDED: resets PID integrators, enters TAKEOFF, climbs to 1.5 m hover height. If airborne (HOVER/SCANNING/GOTO_TARGET/AUTOLAND): enters LANDING_HOME, descends in-place.
C	Terrain Scan	HOVER only	Initiates a 360° yaw spin. During spin, the onboard scanner reveals cells within sensor radius. After full rotation, transitions to HOVER and marks the occupancy grid as ready.
S	Toggle Path Overlay	Any airborne	Shows/hides the green path capsule line. If path shown and target is set: starts GOTO_TARGET flight. If path hidden during TAKEOFF or GOTO_TARGET: immediately transitions to HOVER.
1–5	Select Target Pad	HOVER / LANDED / GOTO_TARGET	Calculates path from current position to selected pad using the active algorithm (A* or Dijkstra). Path is computed but flight does NOT start until S is pressed.
H	Select Home Pad	HOVER / LANDED / GOTO_TARGET	Same as 1–5 but targets the Home pad at (–4, –4).
O	Spawn Obstacle	Any	If GOTO_TARGET: spawns an obstacle 1.2–2.5 m ahead on the planned path. Otherwise: spawns at a random valid coordinate avoiding landing pads.
K	Kill Switch	Any	Immediately cuts all motor thrust via the Priority 5 subsumption behaviour. Clears all path data, resets to LANDED. The drone will fall under gravity.
P	Toggle Algorithm	Any	Switches the pathfinder between A* and Dijkstra. The change applies to the next path calculation.
R	Toggle Random Spawning	Any	Enables/disables periodic random obstacle spawning every 12 seconds. If the drone is flying, obstacles are spawned in front of it on the path.

6. Standard Flight Operations (Step-by-Step)

6.1 Basic Flight to a Pad

Step 1 — Takeoff: Press **L**. The drone climbs from the home pad to 1.5 m hover altitude. Console prints `[CTRL] Takeoff!` and `[CTRL] Hover reached`.

Step 2 — Scan (first flight only): Press **C**. The drone performs a 360° spin, revealing terrain within a 2.5 m radius via its directional sensor. Console prints `[SCAN] 360° scan complete!`.

Step 3 — Select Target: Press a pad key (e.g., **2** for Pad 2). Console prints `[NAV] Planning A* from ... → ...` and shows waypoint count / cost.

Step 4 — Start Flight: Press **S**. The green path line appears and the drone begins following it. The behaviour manager activates `Navigation (Priority 2)`.

Step 5 — Arrival: When the drone reaches within 30 cm of the target pad, it automatically transitions to AUTOLAND → DESCENDING → WAITING (2 s) → LANDED. Path data is cleared.

6.2 Flight with Terrain Discovery

On the very first run, the map starts mostly unexplored (Fog of War). The scanner reveals cells as the drone moves. The planner uses an **optimistic strategy**: unexplored cells are treated as free space. If the drone later discovers an obstacle in an unexplored region while navigating, the subsumption avoidance layer detects the path blockage and automatically replans.

6.3 Flight with Dynamic Obstacle Avoidance

Step 1: Begin a flight to any pad (Steps 1–4 above).

Step 2 — Spawn Obstacle: Press **O** while the drone is in GOTO_TARGET. An obstacle spawns 1.2–2.5 m ahead on the planned path. Console prints `[DYNAMIC] Spawned ...`.

Step 3 — Automatic Replanning: The scanner detects the obstacle at the blocked waypoint. The subsumption avoidance layer (Priority 3) pauses navigation, recalculates the path, rebuilds the waypoints, and resumes flight. Console prints `[AVOIDANCE] Re-routing ...`.

Step 4: The drone seamlessly continues to the target on the new route.

6.4 Using Periodic Random Obstacle Spawning

Press **R** to enable periodic spawning. Every 12 seconds, there is an 80% chance that a new obstacle is generated. If the drone is actively navigating, the obstacle appears directly on its path; otherwise it spawns at a random arena coordinate. Press **R** again to disable. Up to 5 dynamic obstacles can be active simultaneously; the oldest is recycled when all 5 are used.

7. Terrain Scanning System

The terrain scanner (`terrain_scanner.py`) maintains an internal **discovered_grid** and **explored_mask**. When the simulation starts, only the immediate vicinity of all landing pads is pre-revealed. The rest of the 160×160 grid is marked as unknown (value –1).

Sensor Modes:

- **Directional (Default):** A 90° forward-facing cone sensor with 2.5 m range. Cells must be within the FOV and range to be revealed. This models a front-facing depth camera or LiDAR.
- **Omnidirectional:** Reveals all cells within the sensor radius regardless of heading. This models a 360° scanning LiDAR. Can be enabled programmatically.

The drone's yaw heading determines the scan direction. During a 360° spin scan (C key), the directional sensor sweeps the full circle, achieving near-omnidirectional coverage.

Optimistic Planning: The `get_planning_grid()` method returns a grid where all unknown cells (–1) are mapped to free space (0). This means the pathfinder will plan routes through unexplored terrain. If the drone later discovers an obstacle there, the path is invalidated and replanned.

Visual Output: The `ground_map.png` file uses a dark Fog-of-War overlay for unexplored cells, bright red for discovered obstacles, and the neon grid background for explored free space.

---

8. Obstacle Avoidance

Five static MuJoCo bodies (`dynamic_obstacle_1` through `dynamic_obstacle_5`) are defined in `scene.xml` at z = –10 m (underground, invisible). When spawned, they are repositioned to the arena surface using `model.body_pos` and become visible glowing magenta obstacles.

Spawning Methods:

- **Keyboard (O):** Instant manual spawn. If navigating, spawns on path ahead; otherwise random.
- **Periodic Random (R):** Automatic spawn every 12 seconds (80% probability).
- **Programmatic:** Call `obstacle_manager.spawn_obstacle(model, data, x, y, grid)`.

Obstacle Recycling: When all 5 obstacle slots are used and a new spawn is requested, the oldest active obstacle is removed from the occupancy grid and repositioned to the new location (FIFO recycling). Landing pad zones are always protected from obstacle placement.

Grid Updates: Each spawned obstacle marks occupied cells on the ground truth grid using a circular mask. The scanner must then discover these cells for the planner to see them. The viewer draws a translucent magenta ring around each active obstacle's base.

9. Behavior Priorities

The subsumption architecture wraps the existing autopilot controller with a priority-based decision layer. At each simulation step, the BehaviorManager evaluates all behaviours from highest to lowest priority. The first behaviour whose `check_trigger()` returns True gets exclusive control of the drone.

Priority	Behaviour Name	Trigger Condition	Action
5 (Highest)	Kill Switch	kill_switch_active == True	Immediately clears all forces, sets LANDED, clears path data.
4	Emergency Landing	state == LANDING_HOME	Descends in-place using PID controller, transitions to LANDED on ground contact.
3	Obstacle Avoidance	state == GOTO_TARGET AND path_blocked == True	Pauses navigation, recalculates path from current position using active algorithm, resumes.
2	Navigation	state in {TAKEOFF, HOVER, GOTO_TARGET, AUTOLAND}	Normal flight operations: climb, hover, path-follow, autoland at destination pad.
1 (Lowest)	Terrain Scanning	state == SCANNING	Performs 360° yaw spin for terrain mapping, maintains altitude.

Behaviour Override Example: If the drone is scanning (Priority 1) and the kill switch is pressed, the Kill Switch behaviour (Priority 5) takes over immediately, cutting all forces. Similarly, if a path blockage is detected during navigation (Priority 2), the Obstacle Avoidance behaviour (Priority 3) temporarily takes control to replan, then navigation resumes.

---

10. Path Selection (A* / Dijkstra)

Press **P** at any time to toggle between A* and Dijkstra. The currently selected algorithm is used for all subsequent path calculations (both user-commanded and automatic replanning). The console prints the active algorithm after toggling.

Property	A* (Default)	Dijkstra
Heuristic	Octile distance (admissible for 8-connected grid)	None (uniform cost)
Optimality	Optimal (same cost as Dijkstra)	Optimal
Node Expansion	Significantly fewer (directed search)	Explores all reachable nodes
Speed	Faster (especially on large grids)	Slower (exhaustive)
Use Case	Real-time point-to-point navigation	Full-tour path computation

Both algorithms use the same 8-connected grid movement with obstacle inflation cost: cells adjacent to obstacles incur a +3.0 cost penalty to maintain safe clearance.

11. Sensor Filtering

The sensor simulation suite (`sensor_simulation.py`) runs at every physics timestep (200 Hz). It reads ground truth from MuJoCo and injects realistic Gaussian noise.

Sensor	Rate	Noise Std Dev	Frame	Measurement
GPS	10 Hz	XY: 0.05 m, Z: 0.10 m	World	Position (x, y, z)
IMU Accelerometer	200 Hz	0.15 m/s²	Body	Proper acceleration (ax, ay, az)
IMU Gyroscope	200 Hz	0.02 rad/s	Body	Angular velocity (ωx, ωy, ωz)
Barometer	200 Hz	0.10 m	World	Absolute altitude (z)
Altimeter (LiDAR)	200 Hz	0.015 m	Drone-down	Distance to ground
Front Rangefinder	200 Hz	0.03 m	Drone-forward	Distance to nearest obstacle (mj_ray)

Implemented Filters:

- **EMA (α=0.15):** Simple low-pass filter. Good noise reduction, but introduces phase delay.
- **Complementary (α=0.98):** Fuses gyroscope (high-frequency, low drift) with accelerometer (low-frequency, accurate DC) for attitude estimation. Also fuses accelerometer with barometer for altitude.
- **Kalman Filter (2-state):** Optimal kinematic estimator [position, velocity]. Uses acceleration as control input and GPS/barometer as measurements. Best overall RMSE and zero phase delay.

On simulation exit, a 6-panel comparison plot is saved to `sensor_filtering_analysis.png`.

---

12. Output Files

flight_trajectory.csv logs every 20th physics step (~10 Hz effective rate): time, state, X, Y, Z, quaternion (qw, qx, qy, qz), velocities (vx, vy, vz), and applied forces (fx, fy, fz).

detection_log.csv records discrete flight events: TAKEOFF, HOVER_REACHED, SCAN_START, SCAN_COMPLETE, GOTO_TARGET_START, PAD_ARRIVED, TOUCHDOWN, LANDED, EMERGENCY_LAND, REPLAN_SUCCESS, REPLAN_FAILED, KILL_SWITCH_TRIGGERED.

map_data.json is a complete JSON dump of the occupancy grid, landing spot metadata, the current shortest path waypoints, and grid metadata (origin, resolution, dimensions).

13. Error Handling

This section documents how the simulation handles unusual user interactions and boundary conditions. Every scenario has been analyzed by studying the key callback, state machine, and behaviour manager.

Pressing L while LANDED:

Initiates normal takeoff. Safe.

Pressing L while already airborne (HOVER / SCANNING / GOTO_TARGET / AUTOLAND):

Enters LANDING_HOME state. The Emergency Landing behaviour (Priority 4) activates and descends in-place. Path data is cleared. The drone lands at its current X/Y position.

Pressing C while LANDED or during flight (not HOVER):

Ignored. Console prints: `[SCAN] Must be HOVER. Current: <state>`. No state change occurs.

Pressing S with no path calculated:

Console prints: `[VIZ] No path yet.` No state change.

Pressing S to hide path during GOTO_TARGET flight:

Immediately transitions to HOVER. The drone stops mid-flight and holds position.

Pressing a pad key (1-5, H) while SCANNING:

Rejected. `cmd_goto()` requires HOVER, LANDED, or GOTO_TARGET. Console prints the requirement.

Pressing a pad key while already navigating to that pad:

Ignored. Console prints: `[NAV] Already flying to <pad>.`

Pressing a pad key while navigating to a different pad:

Accepted. New path is calculated from the current drone position. Path replaced; old path discarded.

Pressing O when no inactive obstacles remain:

The oldest active obstacle is recycled (FIFO). It is removed from the grid and repositioned.

Pressing K at any time:

The Kill Switch behaviour (Priority 5) immediately fires. All motor thrust is zeroed. All path state is cleared. The drone enters LANDED (it may drop if airborne due to gravity). This is by design — it simulates an emergency motor cutoff.

Pressing K while LANDED:

The kill_switch_active flag is set, but on the next step the LANDED fast-path in the behaviour manager clears forces before the Kill Switch can fire. Harmless — no state change.

Pressing P during active flight:

Algorithm is toggled. The current flight continues on the existing path. The new algorithm is only used for the next planning call (next pad selection or replanning event).

Pressing R during LANDED:

Periodic spawning is enabled, but obstacles only spawn when the timer fires. If the drone is not flying, obstacles spawn at random arena positions.

Multiple rapid key presses:

Each keypress is processed sequentially in the key_callback. The simulation loop runs at 200 Hz, so key events are handled between physics steps. No race conditions.

Path to a blocked/unreachable pad:

The pathfinder returns None. Console prints: `[NAV] A*/Dijkstra failed`. No state change. User may scan more terrain or remove obstacles.

Obstacle spawned exactly on a landing pad:

Protected by pad-avoidance logic. The spawn_random() and _update_grid_obstacle() methods exclude landing pad regions. If forced programmatically, the pad's cells are not overwritten.

Drone flies through an unscanned obstacle:

Possible if the obstacle is in unexplored terrain and the drone's sensor hasn't revealed it yet. In practice, the sensor's 2.5 m range usually detects obstacles before the drone reaches them, especially with the directional 90° FOV pointing forward.

Closing the viewer window:

The main loop exits gracefully. Trajectory logs are flushed, and the sensor filtering analysis graph is generated and saved before Python exits.

14. Troubleshooting Guide

Problem: Viewer opens but keypresses don't work

Solution: Click inside the MuJoCo viewer window first. The window must have OS-level input focus to receive keyboard events.

Problem: 'scene.xml not found' error

Solution: Ensure you run the simulation from the project directory. All file paths are relative.

Problem: 'drone_joint not found' error

Solution: The scene.xml file may be corrupted. Re-run `add_rocks.py` to regenerate the dynamic stones block. Ensure the drone body is intact.

Problem: Drone oscillates or flips during hover

Solution: Check if the drone mass is correct (expected ~1.8 kg). The PID gains are tuned for this mass. If scene.xml was modified, the gains may need adjustment.

Problem: Path planning fails — 'no path found'

Solution 1: The destination may be surrounded by obstacles. Try scanning more terrain first (C key). **Solution 2:** Dynamic obstacles may have blocked all routes. Wait for some to be recycled.

Problem: Dynamic obstacles don't appear visually

Solution: The obstacle bodies may not have been loaded from scene.xml. Ensure `dynamic_obstacle_1` through `dynamic_obstacle_5` exist in the XML file inside `<worldbody>`.

Problem: sensor_filtering_analysis.png is not generated

Solution: The graph is generated only when the simulation exits normally (close the viewer or Ctrl+C). Force-killing the process skips the cleanup. Ensure matplotlib is installed.

Problem: ImportError for terrain_scanner, dynamic_obstacles, or subsumption

Solution: These files must be in the same directory as simulate.py. Verify all .py files are present.

Problem: ReportLab not found when generating PDFs

Solution: Install it: `pip install reportlab`.

15. Appendix

The drone autopilot operates as a finite state machine. The following table documents every valid state, its entry conditions, active behaviour, and exit transitions.

State	Description	Active Behaviour	Transitions To
LANDED	Motors off, on ground	None (idle)	TAKEOFF (L key)
TAKEOFF	Climbing to hover height	Navigation (P2)	HOVER (reached altitude or path hidden)
HOVER	Holding position at altitude	Navigation (P2)	SCANNING (C key), GOTO_TARGET (S key + target), LANDING_HOME (L key)
SCANNING	360° yaw spin for mapping	Terrain Scanning (P1)	HOVER (scan complete)
GOTO_TARGET	Following planned path	Navigation (P2) or Avoidance (P3)	AUTOLAND (arrived), HOVER (S key / path lost)
AUTOLAND	Descending onto target pad	Navigation (P2)	LANDED (touchdown + 2s wait)
LANDING_HOME	Emergency descent in-place	Emergency Landing (P4)	LANDED (ground contact)

Note: *The Kill Switch (K key) can transition from ANY state to LANDED instantly by activating the Priority 5 behaviour. This overrides all other transitions.*